SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE	DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Program Name: B. Tech	Assignment Type: Lab	Academic Year:2025-2026

Course Coordinator Name	Dr. Rishabh Mittal
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju

CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week9 – Monday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Holl.NO:	2303A54064	**Batche:**	47b

Assignment Number: 18.1(Present assignment number)/**24**(Total number of assignments)

Q.No.	Question	*Expected Time to complete*
1	Lab 18 – API Integration: Connecting to External Services with Error Handling	Week9 - Monday

	Lab Objectives: • Learn how to integrate Python programs with external REST APIs. • Understand API request/response handling using requests or similar libraries. • Implement proper error handling for failed API calls (timeouts, invalid responses, rate limits). • Practice extracting and displaying meaningful information from API responses.	
	Task 1 – Weather API Integration Connect to a Weather API to fetch current weather details for a given city. Instructions: • Prompt AI to generate Python code using the requests library. • Handle errors: invalid city name, API key errors, and network failures. • Display temperature, humidity, and weather description in a readable format. Expected Output: • A working Python script that fetches and displays weather data with proper error handling.	

ai_ass.18.1.py > task.01.py > ...

```python
import requests

# ============================
# CONFIGURATION
# ============================
API_KEY = "YOUR_API_KEY_HERE"   # Replace with your API key
BASE_URL = "https://api.openweathermap.org/data/2.5/weather"


def get_weather(city):
    """Fetch weather data for a given city"""

    params = {
        "q": city,
        "appid": API_KEY,
        "units": "metric"   # Celsius
    }

    try:
        # Send request
        response = requests.get(BASE_URL, params=params, timeout=10)

        # Check HTTP errors
        response.raise_for_status()

        data = response.json()

        # Handle API errors
        if data.get("cod") != 200:
            print(f"❌ Error: {data.get('message', 'Unknown error')}")
            return

        # Extract required information
        temperature = data["main"]["temp"]
        humidity = data["main"]["humidity"]
        description = data["weather"][0]["description"].title()

        # Display results
        print("\n🌤 Weather Report")
        print("----------------------------")
        print(f"City        : {city}")
        print(f"Temperature : {temperature}°C")
        print(f"Humidity    : {humidity}%")
        print(f"Condition   : {description}")
        print("----------------------------")

    except requests.exceptions.HTTPError:
        print("❌ Invalid city name or API issue.")

    except requests.exceptions.ConnectionError:
        print("❌ Network error. Check your internet connection.")

    except requests.exceptions.Timeout:
        print("❌ Request timed out.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

TERMINAL

```
Enter city name: warangal
❌ Invalid city name or API issue.
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding>
```

Task 2 – Currency Exchange Rates

Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.

Instructions:

- Use AI to generate API request code.
- Validate user input for amount and currency codes.
- Implement error handling for invalid responses and API downtime.
- Show conversion results clearly in tabular format.

Expected Output:

- A script that converts INR to multiple currencies with graceful error handling.

ai_ass.18.1.py > task.02.py > ...

```python
import requests

# ==============================
# CONFIGURATION
# ==============================
API_KEY = "YOUR_API_KEY_HERE"   # Replace with your API key
BASE_URL = f"https://v6.exchangerate-api.com/v6/{API_KEY}/latest/INR"

SUPPORTED_CURRENCIES = ["USD", "EUR", "GBP"]


# ==============================
# VALIDATION FUNCTIONS
# ==============================
def validate_amount(amount):
    try:
        value = float(amount)
        if value <= 0:
            raise ValueError
        return value
    except ValueError:
        print("❌ Invalid amount. Enter a positive number.")
        return None


# ==============================
# FETCH EXCHANGE RATES
# ==============================
def fetch_rates():
    try:
        response = requests.get(BASE_URL, timeout=10)
        response.raise_for_status()

        data = response.json()

        # API error handling
        if data["result"] != "success":
            print("❌ API Error:", data.get("error-type"))
            return None

        return data["conversion_rates"]

    except requests.exceptions.ConnectionError:
        print("❌ Network error. Check internet connection.")
    except requests.exceptions.Timeout:
        print("❌ Request timed out.")
    except requests.exceptions.RequestException as e:
        print("❌ Unexpected error:", e)

    return None


# ==============================
# DISPLAY TABLE
```

ai_ass.18.1.py > task.02.py > ...

```python
def fetch_rates():
        print("❌ Network error. Check internet connection.")
    except requests.exceptions.Timeout:
        print("❌ Request timed out.")
    except requests.exceptions.RequestException as e:
        print("❌ Unexpected error:", e)

    return None


# ==============================
# DISPLAY TABLE
# ==============================
def display_conversion(amount, rates):
    print("\n💱 Currency Conversion Results")
    print("=" * 40)
    print(f"{'Currency':<10}{'Rate':<12}{'Converted Amount'}")
    print("-" * 40)

    for currency in SUPPORTED_CURRENCIES:
        rate = rates.get(currency)
        if rate:
            converted = amount * rate
            print(f"{currency:<10}{rate:<12.4f}{converted:.2f}")

    print("=" * 40)


# ==============================
# MAIN PROGRAM
# ==============================
def main():
    amount_input = input("Enter amount in INR: ")

    amount = validate_amount(amount_input)
    if amount is None:
        return

    rates = fetch_rates()
    if rates is None:
        return

    display_conversion(amount, rates)


if __name__ == "__main__":
    main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

TERMINAL

```
Enter amount in INR: 1234567
❌ Unexpected error: 403 Client Error: Forbidden for url: https://v6.exc
```

Task 3 – GitHub Repository Info Fetcher

Connect to the GitHub API to fetch details of a repository (e.g., stars,

forks, issues).

Instructions:

- Prompt AI to write a Python function to send GET requests to GitHub API.
- Handle rate limit errors, 404 (repo not found), and invalid input.
- Display repository name, description, stars, forks, and open issues.

Expected Output:

- A Python program that retrieves and displays GitHub repository information with robust error handling.

ai_ass.16.1.py > ❖ task.03.py > ...

```python
import requests

# ================================
# FUNCTION TO FETCH REPO INFO
# ================================
def fetch_github_repo(owner, repo):
    """
    Fetch repository details from GitHub API
    """

    # Validate input
    if not owner or not repo:
        print("❌ Invalid input. Owner and repository name required.")
        return

    url = f"https://api.github.com/repos/{owner}/{repo}"

    try:
        response = requests.get(url, timeout=10)

        # ---- Rate limit handling ----
        if response.status_code == 403:
            if "rate limit" in response.text.lower():
                print("❌ GitHub API rate limit exceeded. Try again later.")
                return

        # ---- Repository not found ----
        if response.status_code == 404:
            print("❌ Repository not found.")
            return

        # Raise other HTTP errors
        response.raise_for_status()

        data = response.json()

        # Extract required fields
        repo_name = data.get("full_name", "N/A")
        description = data.get("description", "No description")
        stars = data.get("stargazers_count", 0)
        forks = data.get("forks_count", 0)
        open_issues = data.get("open_issues_count", 0)
```

ai_ass.18.1.py > task.03.py > ...

```python
def fetch_github_repo(owner, repo):
        open_issues = data.get("open_issues_count", 0)

        # Display output
        print("\n📦 GitHub Repository Information")
        print("=" * 45)
        print(f"Repository : {repo_name}")
        print(f"Description: {description}")
        print(f"⭐ Stars    : {stars}")
        print(f"🍴 Forks    : {forks}")
        print(f"🐞 Open Issues: {open_issues}")
        print("=" * 45)

    except requests.exceptions.ConnectionError:
        print("❌ Network error. Check internet connection.")

    except requests.exceptions.Timeout:
        print("❌ Request timed out.")

    except requests.exceptions.RequestException as e:
        print("❌ Unexpected error:", e)


# ================================
# MAIN PROGRAM
# ================================
def main():
    print("GitHub Repository Info Fetcher")

    owner = input("Enter repository owner (username/org): ").strip()
    repo = input("Enter repository name: ").strip()

    fetch_github_repo(owner, repo)


if __name__ == "__main__":
    main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

TERMINAL

```
GitHub Repository Info Fetcher
Enter repository owner (username/org): python
Enter repository name: cpython

📦 GitHub Repository Information
📦 GitHub Repository Information
=============================================
Repository : python/cpython
Description: The Python programming language
⭐ Stars    : 72114
🍴 Forks    : 34329
🐞 Open Issues: 9342
=============================================
```

Task 4 – Real-Time Application: News Headlines Aggregator

Scenario: Build a News Aggregator using a free News API.

Requirements:

- Fetch top 5 headlines for a given category (sports, technology, health).
- Handle errors like invalid category, missing API key, and request failures.
- Display headlines in a user-friendly numbered list format.
- Include a retry mechanism if the first request fails.

Expected Output:

- A working Python script that retrieves and displays news headlines with proper error handling

ai_ass.18.1.py > task.04.py > ...

```python
import requests
import time

# ==================================
# CONFIGURATION
# ==================================
API_KEY = "YOUR_API_KEY_HERE"   # Replace with your API key
BASE_URL = "https://newsapi.org/v2/top-headlines"

VALID_CATEGORIES = ["sports", "technology", "health"]
MAX_RETRIES = 3


# ==================================
# VALIDATE CATEGORY
# ==================================
def validate_category(category):
    if category.lower() not in VALID_CATEGORIES:
        print(f"❌ Invalid category! Choose from: {', '.join(VALID_CATEGORIES)}")
        return None
    return category.lower()


# ==================================
# FETCH NEWS WITH RETRY MECHANISM
# ==================================
def fetch_news(category):
    params = {
        "apiKey": API_KEY,
        "category": category,
        "country": "us",
        "pageSize": 5
    }

    attempt = 0

    while attempt < MAX_RETRIES:
        try:
            response = requests.get(BASE_URL, params=params, timeout=10)

            # API errors
            if response.status_code == 401:
                print("❌ Missing or invalid API key.")
                return None

            if response.status_code != 200:
                raise requests.exceptions.RequestException(
                    f"HTTP {response.status_code}"
                )

            data = response.json()

            if data["status"] != "ok":
                print("❌ API Error:", data.get("message"))
                return None

            return data["articles"]

        except requests.exceptions.ConnectionError:
            print(f"⚠ Network error (Attempt {attempt+1}/{MAX_RETRIES})")

        except requests.exceptions.Timeout:
            print(f"⚠ Request timed out (Attempt {attempt+1}/{MAX_RETRIES})")

        except requests.exceptions.RequestException as e:
```

```python
def fetch_news(category):
        except requests.exceptions.Timeout:
            print(f"⚠ Request timed out (Attempt {attempt+1}/{MAX_RETRIES})")

        except requests.exceptions.RequestException as e:
            print(f"⚠ Request failed: {e}")

        attempt += 1
        time.sleep(2)  # wait before retry

    print("❌ Failed after multiple retry attempts.")
    return None


# ==================================
# DISPLAY HEADLINES
# ==================================
def display_headlines(articles, category):
    print(f"\n📰 Top 5 {category.capitalize()} Headlines")
    print("=" * 45)

    for i, article in enumerate(articles, start=1):
        title = article.get("title", "No title available")
        print(f"{i}. {title}")

    print("=" * 45)


# ==================================
# MAIN PROGRAM
# ==================================
def main():
    print("News Headlines Aggregator")
    print("Available categories:", ", ".join(VALID_CATEGORIES))

    category_input = input("Enter category: ")
    category = validate_category(category_input)

    if not category:
        return

    articles = fetch_news(category)

    if articles:
        display_headlines(articles, category)


if __name__ == "__main__":
    main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

TERMINAL

```
News Headlines Aggregator
Available categories: sports, technology, health
Enter category: technology
❌ Missing or invalid API key.
```